Introduction to Artificial Intelligence and Machine Learning (CSN11601)

Assignment-8

Name : Aayansh Niranjan Date : 15/11/2025
Reg_No.: 20253004

Question 1: Spam Email Classification

You are given a dataset of emails labeled as 'spam' or 'not spam'. The features represent word counts or term frequencies (bag-of-words) for a vocabulary of around 1000 words. The goal is to automatically detect spam messages using Naive Bayes.

Tasks:

1. Choose the appropriate Naive Bayes variant (e.g., MultinomialNB) and justify your choice.
2. Preprocess the text data by tokenizing, cleaning, and vectorizing it using CountVectorizer or TF-IDF.
3. Train the Naive Bayes model, make predictions on a test set, and compute evaluation metrics such as accuracy, precision, recall, and F1-score.
4. Discuss how your model would handle words that appear in the test set but not in training, and propose smoothing techniques (e.g., Laplace smoothing).
5. (Bonus) Improve model performance using stopword removal, feature selection, or TF-IDF weighting.

A1. CODE:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB

data = pd.read_csv("SMSSpamCollection", sep="\t", header=None, names=["label", "message"], encoding="latin-1")
data.columns = ['label', 'message']
data['label'] = data['label'].map({'spam': 1, 'ham': 0})

X_train, X_test, y_train, y_test = train_test_split(data['message'], data['label'], test_size=0.2, random_state=42)

vectorizer = TfidfVectorizer(stop_words='english', max_features=1000)
X_train_tfidf = vectorizer.fit_transform(X_train)
X_test_tfidf = vectorizer.transform(X_test)

model = MultinomialNB(alpha=1.0)
model.fit(X_train_tfidf, y_train)

user_message = input("Enter an email message: ")
user_message_tfidf = vectorizer.transform([user_message])
prediction = model.predict(user_message_tfidf)

if prediction[0] == 1:
    print("This message is classified as: SPAM")
else:
    print("This message is classified as: NOT SPAM")
```

OUTPUT:

```
(venv) aayansh207@aayansh-PC:~/Desktop/MNNIT/Naive_Bayes$ python3 p1.py
Enter an email message: send the OTP
This message is classified as: NOT SPAM
(venv) aayansh207@aayansh-PC:~/Desktop/MNNIT/Naive_Bayes$ python3 p1.py
Enter an email message: Win Free CAsh
This message is classified as: SPAM
```

Question 2: Predicting Customer Churn (Continuous + Categorical Features)

A telecom company provides you a dataset where each record corresponds to a customer. Features include monthly usage (continuous), number of calls (continuous), contract type (categorical: 'month-to-month', 'one-year', 'two-year'), and whether they have an international plan (binary). The target variable is churn = yes/no.

Your task is to predict customer churn using Naive Bayes.

Tasks:

1. Decide how to handle mixed continuous and categorical features under Naive Bayes (e.g., use GaussianNB for continuous, CategoricalNB for discrete).
2. Implement preprocessing — discretize continuous variables or assume Gaussian distribution as needed.
3. Train and evaluate your Naive Bayes model using confusion matrix and AUC score.
4. Compare the Naive Bayes model's performance with a logistic regression baseline model.
5. Discuss which features might violate the independence assumption and how that could affect the model's results.

A2. CODE:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.naive_bayes import GaussianNB

data = pd.read_csv("churn.csv")
if 'customerID' in data.columns:
    data = data.drop('customerID', axis=1)
if 'TotalCharges' in data.columns:
    data['TotalCharges'] = pd.to_numeric(data['TotalCharges'], errors='coerce')
numeric_cols = data.select_dtypes(include=np.number).columns
data[numeric_cols] = data[numeric_cols].fillna(data[numeric_cols].mean())

X = data.drop('Churn', axis=1)
y = data['Churn']

label_encoders = {}
for col in X.columns:
    if X[col].dtype == 'object' or X[col].dtype.name == 'category':
        le = LabelEncoder()
        X[col] = le.fit_transform(X[col].astype(str))
        label_encoders[col] = le

target_encoder = LabelEncoder()
y = target_encoder.fit_transform(y.astype(str))

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)

model = GaussianNB()
model.fit(X_train_scaled, y_train)
```

```
print("Enter new customer details to predict churn:")
input_data = {}
for col in X.columns:
    if col in label_encoders:
        options = list(label_encoders[col].classes_)
        print(f"{col} options: {options}")
        val = input(f"Enter {col}: ")
        val_str = str(val)
        if val_str not in label_encoders[col].classes_:
            label_encoders[col].classes_ = np.append(label_encoders[col].classes_, val_str)
        input_data[col] = int(label_encoders[col].transform([val_str])[0])
    else:
        val = input(f"Enter {col} (numeric): ")
        input_data[col] = float(val)

user_df = pd.DataFrame([input_data], columns=X.columns)
user_scaled = scaler.transform(user_df)
pred = model.predict(user_scaled)

if pred[0] == 1:
    print("\nPrediction: The customer is likely to CHURN.")
else:
    print("\nPrediction: The customer is NOT likely to churn.")
```

OUTPUT:

```
(venv) aayansh207@aayansh-PC:~/Desktop/MNNIT/Naive_Bayes$ python3 p2.py
Enter new customer details to predict churn:
gender options: ['Female', 'Male']
Enter gender: Male
Enter SeniorCitizen (numeric): 0
Partner options: ['No', 'Yes']
Enter Partner: Yes
Dependents options: ['No', 'Yes']
Enter Dependents: 2
Enter tenure (numeric): 4
PhoneService options: ['No', 'Yes']
Enter PhoneService: 7987954654
MultipleLines options: ['No', 'No phone service', 'Yes']
Enter MultipleLines: Yes
InternetService options: ['DSL', 'Fiber optic', 'No']
Enter InternetService: DSL
OnlineSecurity options: ['No', 'No internet service', 'Yes']
Enter OnlineSecurity: No
OnlineBackup options: ['No', 'No internet service', 'Yes']
Enter OnlineBackup: Yes
DeviceProtection options: ['No', 'No internet service', 'Yes']
Enter DeviceProtection: No
TechSupport options: ['No', 'No internet service', 'Yes']
Enter TechSupport: No
StreamingTV options: ['No', 'No internet service', 'Yes']
Enter StreamingTV: Yes
StreamingMovies options: ['No', 'No internet service', 'Yes']
Enter StreamingMovies: Yes
Contract options: ['Month-to-month', 'One year', 'Two year']
Enter Contract: Two year
PaperlessBilling options: ['No', 'Yes']
Enter PaperlessBilling: No
PaymentMethod options: ['Bank transfer (automatic)', 'Credit card (automatic)', 'Electronic check', 'Mailed check']
Enter PaymentMethod: Mailed check
Enter MonthlyCharges (numeric): 1000
Enter TotalCharges (numeric): 1500

Prediction: The customer is NOT likely to churn.
```

Question 3: Sentiment Classification of Movie Reviews

You are given a dataset of movie reviews labeled as positive or negative. Your goal is to build a Naive Bayes classifier that predicts the sentiment of a given movie review.

Tasks:

1. Choose a Naive Bayes variant (MultinomialNB or BernoulliNB) and justify your selection.
2. Preprocess text (tokenize, lowercase, remove punctuation, and optionally apply stemming or lemmatization).
3. Convert text into numerical features using CountVectorizer or TF-IDF representation.
4. Train, test, and evaluate the model using metrics such as accuracy, precision, recall, and F1-score.
5. Handle the inclusion of n-grams (e.g., bigrams) as features to improve model context understanding.
6. (Bonus) Compare performance using unigrams vs. bigrams and with/without smoothing techniques.

A3. CODE:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB

data = pd.read_csv("movies.csv", encoding='utf-8', on_bad_lines='skip')

print(data.head())

if 'review' not in data.columns or 'sentiment' not in data.columns:
    data.columns = ['review', 'sentiment']

X_train, X_test, y_train, y_test = train_test_split(
    data['review'], data['sentiment'], test_size=0.2, random_state=42
)

vectorizer = TfidfVectorizer(stop_words='english', max_features=5000)
X_train_tfidf = vectorizer.fit_transform(X_train)

model = MultinomialNB()
model.fit(X_train_tfidf, y_train)

review = input("Enter a movie review: ")
review_tfidf = vectorizer.transform([review])
prediction = model.predict(review_tfidf)

print("Predicted Sentiment:", prediction[0].upper())
```

OUTPUT:

```
(venv) aayansh207@aayansh-PC:~/Desktop/MNNIT/Naive_Bayes$ python3 p3.py
                                      review sentiment
0  One of the other reviewers has mentioned that ...  positive
1  A wonderful little production. <br /><br />The...  positive
2  I thought this was a wonderful way to spend ti...  positive
3  Basically there's a family where a little boy ...  negative
4  Petter Mattei's "Love in the Time of Money" is...  positive
Enter a movie review: I absolutely loved Coolie! For me, this is a 5-star film all the way. To be honest, it felt more like a Lokesh Kanagaraj movie than a typical Rajini sir mass ente
rtainer, but that was part of the magic. Lokesh’s signature style, the layered screenplay, and all the twists and turns kept me hooked from start to finish.
Predicted Sentiment: POSITIVE
```

Question 4: Disease Prediction from Medical Records

You are provided with a medical dataset containing patient records. Features include age (continuous), blood pressure (continuous), cholesterol level (continuous), smoking status (binary), family history (binary), and BMI category (categorical: underweight/normal/overweight). Your goal is to predict whether a patient has a particular disease using Naive Bayes.

Tasks:

1. Determine how to handle continuous features (GaussianNB) and categorical features (CategoricalNB).
2. Preprocess the data (scaling, encoding, or discretization as required).
3. Train the Naive Bayes model and evaluate its performance using metrics such as accuracy, precision, recall, and F1-score.
4. If some feature distributions are highly skewed (e.g., cholesterol), apply transformations or use non-parametric methods.
5. Discuss how correlated features (e.g., age and BMI) may violate the independence assumption, and suggest mitigation approaches (e.g., feature selection).

A3. CODE:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.naive_bayes import GaussianNB

data = pd.read_csv("diabetes.csv")

X = data.drop('Outcome', axis=1)
y = data['Outcome']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)

model = GaussianNB()
model.fit(X_train_scaled, y_train)

pregnancies = int(input("Enter number of pregnancies: "))
glucose = float(input("Enter glucose level: "))
bp = float(input("Enter blood pressure: "))
skin = float(input("Enter skin thickness: "))
insulin = float(input("Enter insulin level: "))
bmi = float(input("Enter BMI: "))
dpf = float(input("Enter Diabetes Pedigree Function: "))
age = int(input("Enter age: "))

user_input = pd.DataFrame([[pregnancies, glucose, bp, skin, insulin, bmi, dpf, age]],
columns=X.columns)
user_input_scaled = scaler.transform(user_input)

prediction = model.predict(user_input_scaled)

if prediction[0] == 1:
    print("The patient is likely to have the disease.")
else:
    print("The patient is NOT likely to have the disease.")
```

OUTPUT:

```
(venv) aayansh207@aayansh-PC:~/Desktop/MNNIT/Naive_Bayes$ python3 p4.py
Enter number of pregnancies: 2
Enter glucose level: 110
Enter blood pressure: 72
Enter skin thickness: 4
Enter insulin level: 115
Enter BMI: 28
Enter Diabetes Pedigree Function: 2
Enter age: 20
The patient is likely to have the disease.
```